Défis en Intelligence Artificielle. Reinforcement Learning (2)

Xavier Siebert

MathRO, Université de Mons

http://www.ig.fpms.ac.be/~siebertx





Menu

1 Comments on previous session

2 Reinforcement Learning

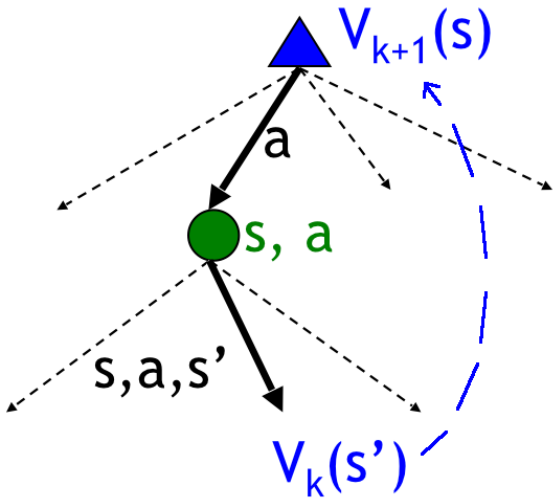
3 Model-based Reinforcement Learning

4 Model-free Reinforcement Learning

Summary : Value Iteration for an MDP

- value function associates to any state $s \in \mathcal{S}$ the value of the expected (deprecated) reward from s.
- Bellman's equation

$$v_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma v_k(s')]$$



Possible implementation of Value Iteration

```
def __init__(self, mdp, discount = 0.9, iterations = 100):
    self.mdp = mdp
    self.discount = discount
    self.iterations = iterations
    self.values = util.Counter()

    self.best_action_dict = {}
    for i in range(iterations):
        nextvalues = self.values.copy()  # version with 2 tables (k and k+1)
        for s in mdp.getStates():
            actions = mdp.getPossibleActions(s)
            if len(actions) > 0:
                bestaction = None
                bestscore = -sys.float_info.max
                for a in actions:
                    E = self.getQValue(s,a)
                    if E > bestscore:
                        bestscore = E
                        bestaction = a
                self.best_action_dict[s] = bestaction
                nextvalues[s] = bestscore
        self.values = nextvalues
```

Possible implementation of getQValue

```
def getValue(self, s):
   return self.values[s]

def getQValue(self, s, a):
    stateprobPaires = self.mdp.getTransitionStatesAndProbs(s, a)
    #return a list of pairs (s', T(s,a,s'))
    #compute the expectation E
    E = 0.0
    for ss, T in stateprobPaires:
        E += T * (self.mdp.getReward(s, a, ss) + self.discount*self.values[ss])
    return E

def getPolicy(self, s):
    return self.best_action_dict.get(s)
```

Menu

1 Comments on previous session

2 Reinforcement Learning

3 Model-based Reinforcement Learning

4 Model-free Reinforcement Learning

Reinforcement Learning (RL)

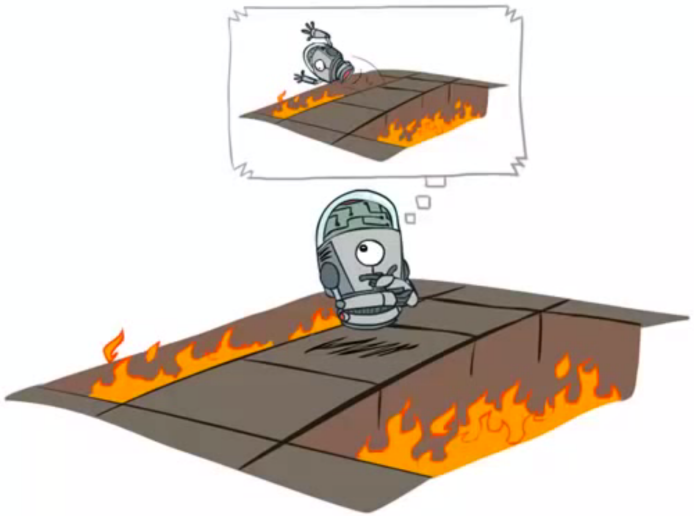
- we are now ready to tackle reinforcement learning (RL)
- main difference with MDP : dynamics of the system unknown
  - $T(s, a, s') = ?$
  - $R(s, a, s') = ?$
- cannot use Bellman's equations as such

$$v^{\pi}(s) = \max_{a} \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma v^{\pi}(s')]$$

- several methods exist to overcome this problem
- we will first classify them in 2 groups
  - model-based RL
  - model-free RL

Model-based RL vs. Model-free RL

- model-based ("indirect", "offline") reinforcement learning
  - Build a model of $\hat{T}(s, a, s')$ and $\hat{R}(s, a, s')$ from **prior** experience
  - "solve MDP" (value iteration, policy iteration, ...) for planning
- model-free ("direct", "online") reinforcement learning
  - no models of $T(s, a, s')$ and $R(s, a, s')$
  - learn progressively from experience



Model-based (offline)



Model-free (online)

Source : ai.berkeley.edu

Menu

1. Comments on previous session
2. Reinforcement Learning
3. Model-based Reinforcement Learning
   - With value iteration
   - With policy iteration
4. Model-free Reinforcement Learning

Model-based RL with value iteration

1. Assume $\mathcal{S}$ and $\mathcal{A}$ are known
2. build a model of $\hat{T}(s, a, s')$ and $\hat{R}(s, a, s')$
3. Solve MDP by value iteration

$$v_{k+1}(s) \leftarrow \max_a \sum_{s'} \hat{T}(s, a, s')[\hat{R}(s, a, s') + \gamma v_k(s')]$$

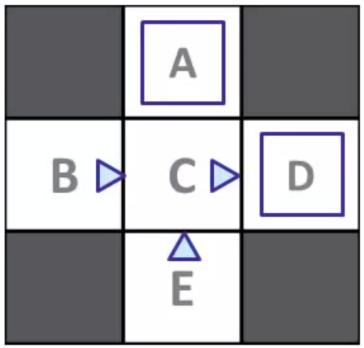
starting from initial values, for example

$$v_0(s) = 0 \quad \forall s \in \mathcal{S}$$

Model-based RL

Example of model building

- example, where terminal states are
  - $D = +10$ (good)
  - $A = -1$ (bad)

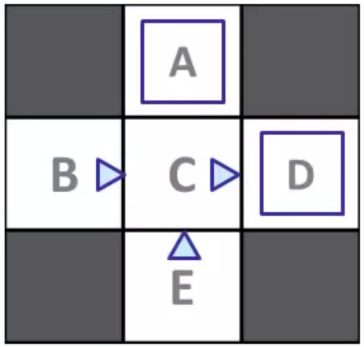


- input policy π
- assume $\gamma = 1$

- construct episodes of training
  1. start from an initial state s
  2. perform an action a
  3. observe the final state s' and the reward $R(s, a, s')$

Model-based RL

Example of model building



- observed episodes of training

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

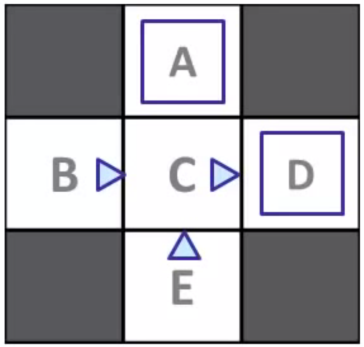
Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Model-based RL

Example of model building

- observed episodes of training



Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

- build a model of $\hat{T}$ and $\hat{R}$ from these observations

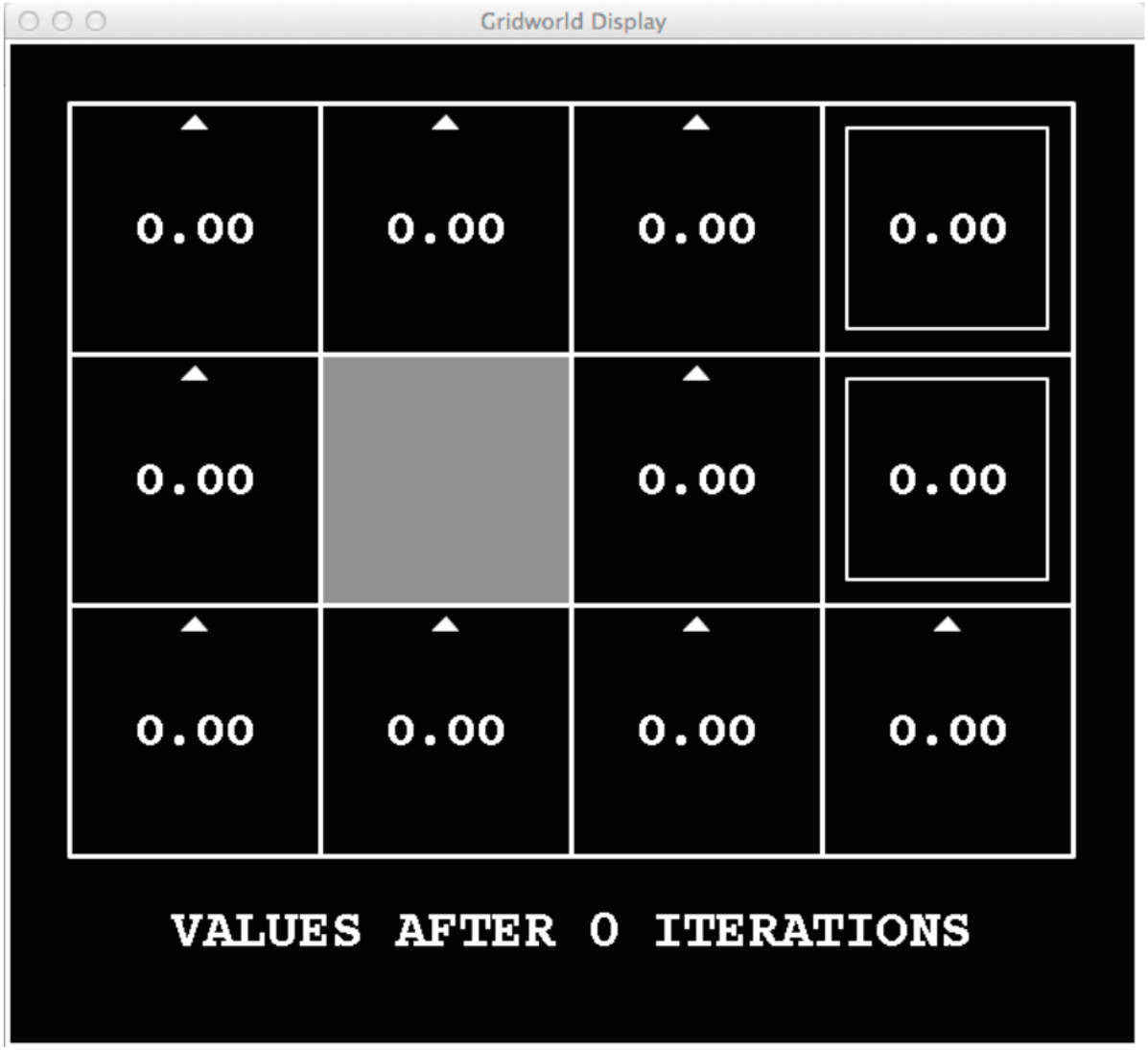
$$\hat{T}(s, a, s')$$

T(B, east, C) = 1.00
T(C, east, D) = 0.75
T(C, east, A) = 0.25
...

$$\hat{R}(s, a, s')$$

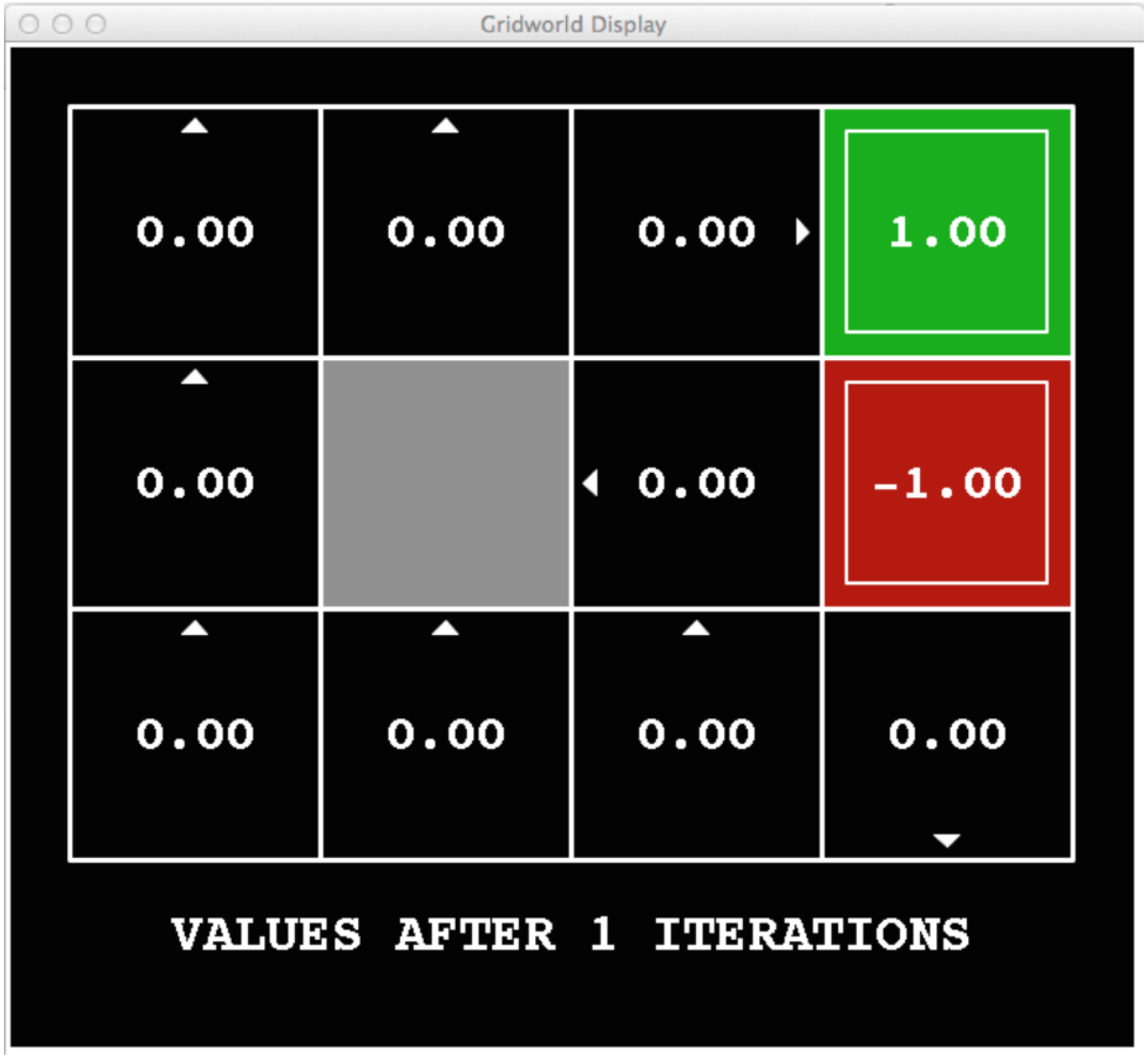
R(B, east, C) = -1
R(C, east, D) = -1
R(D, exit, x) = +10
...

Example : Value Iteration RL with Grid World



Noise = 0.2
Discount = 0.9
Living reward = 0

Example : Value Iteration RL with Grid World



Noise = 0.2
Discount = 0.9
Living reward = 0

Example : Value Iteration RL with Grid World



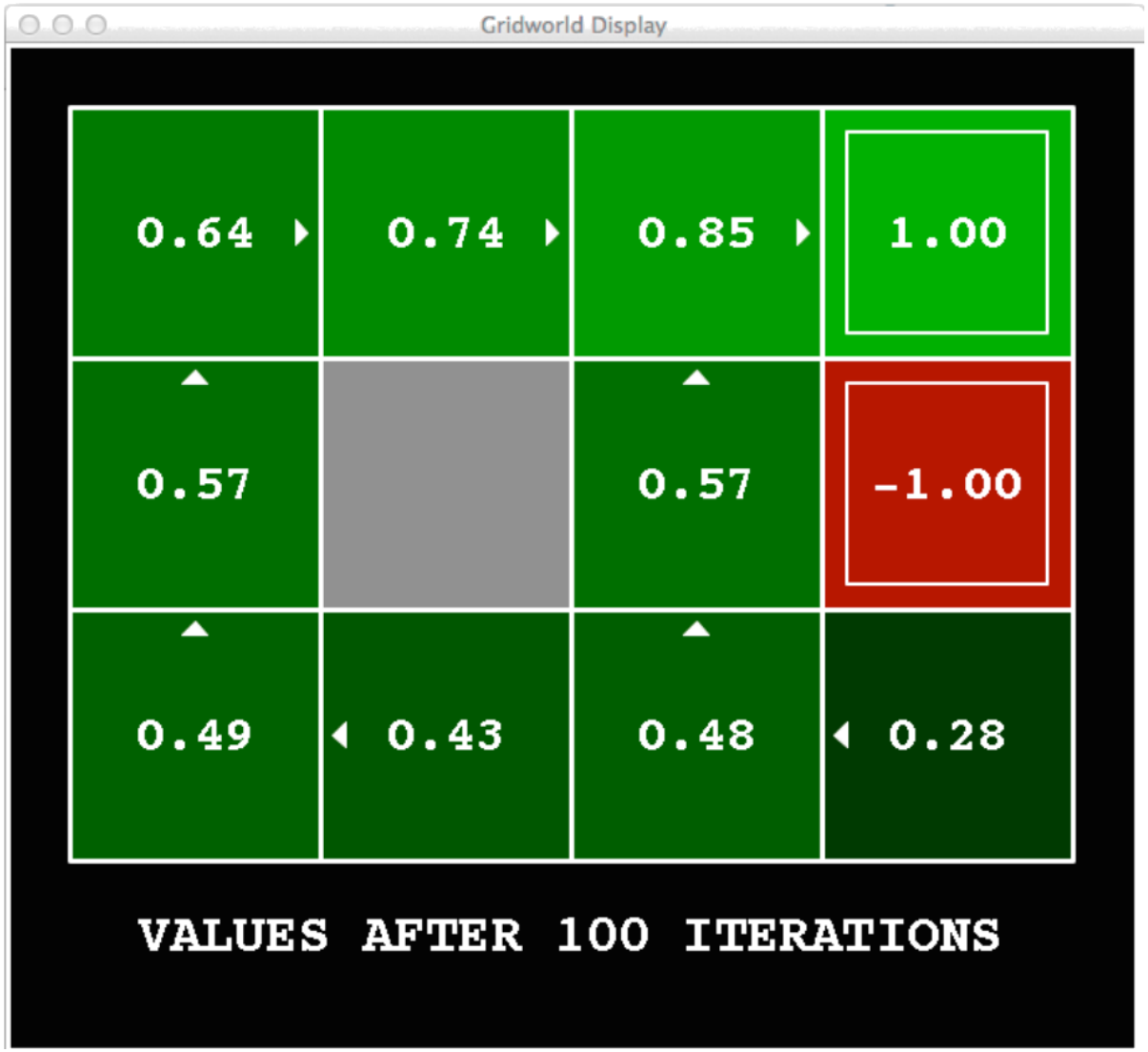
Noise = 0.2
Discount = 0.9
Living reward = 0

Example : Value Iteration RL with Grid World



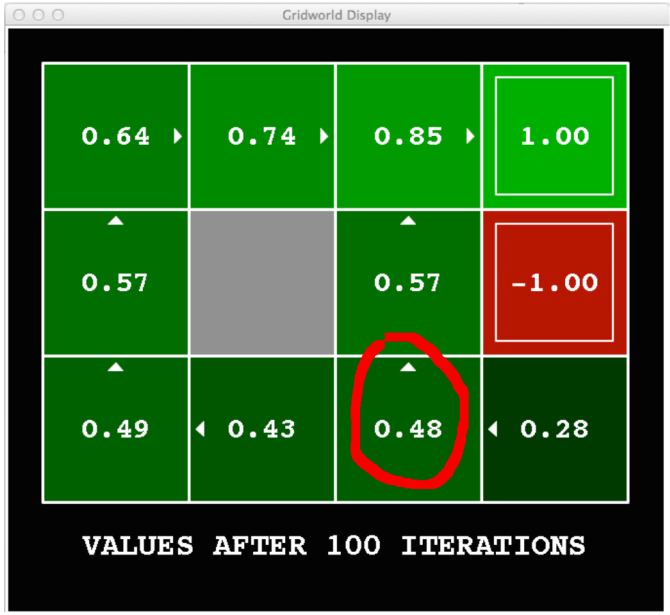
Noise = 0.2
Discount = 0.9
Living reward = 0

Example : Value Iteration RL with Grid World



Noise = 0.2
Discount = 0.9
Living reward = 0

Example : Value Iteration RL with Grid World



Noise = 0.2
Discount = 0.9
Living reward = 0

- best policy : "go left around the gray square"
- after 100 iterations : not quite optimal
- all actions a from all states s to all states s' must be tried : $O(S^2A)$

Model-based RL with policy iteration

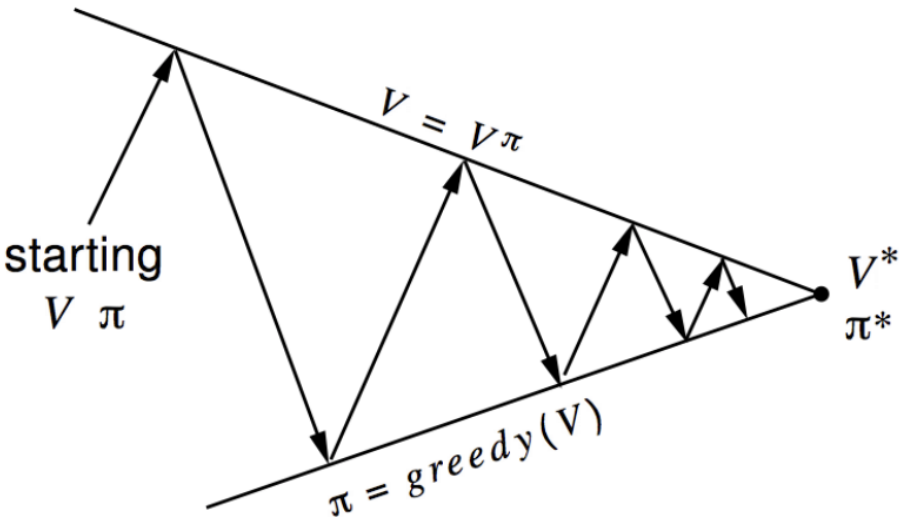
- build approximations $\hat{T}(s, a, s')$ and $\hat{R}(s, a, s')$
- solve MDP by policy iteration

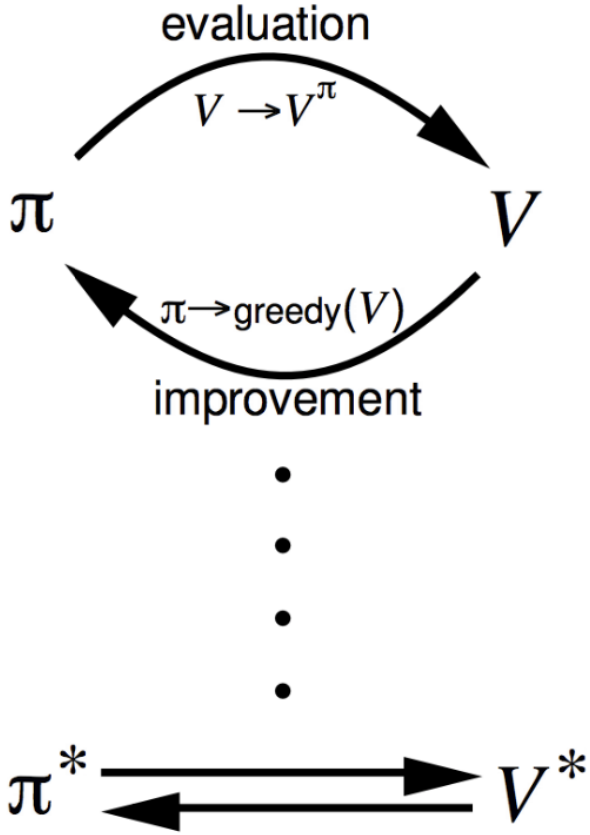
$$v^{\pi}_{k+1}(s) \leftarrow \sum_{s'} \hat{T}(s, \pi(s), s')[\hat{R}(s, \pi(s), s') + \gamma v^{\pi}_{k}(s')]$$

- convergence

$$\lim_{k \to \infty} v^{\pi}_{k}(s) = v^{*}(s)$$

Model-based RL with policy iteration



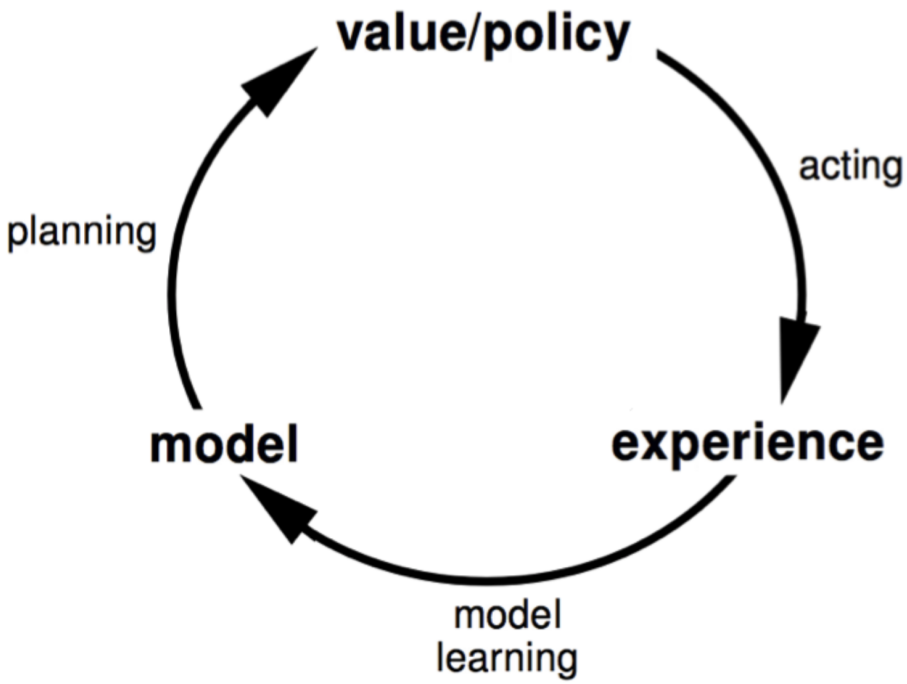


Source : David Silver

Policy evaluation Estimate v_π
e.g. Iterative policy evaluation

Policy improvement Generate $\pi' \geq \pi$
e.g. Greedy policy improvement

Summary of Model-based RL



Summary of Model-based RL

- Advantages
  - A model could give a more compact / useful representation
  - Can efficiently learn model by supervised learning methods
  - Can reason about model uncertainty
- Drawbacks
  - Two sources of approximation error
    1. learn a model
    2. construct a value function

Menu

1 Comments on previous session

2 Reinforcement Learning

3 Model-based Reinforcement Learning

4 Model-free Reinforcement Learning
- Monte-Carlo
- Temporal Differences
- SARSA
- q-learning
- Exploration and Exploitation

Motivations for Model-free RL (1)

- The interesting quantity is the action a, not the precise value $v^*(s)$
- The policy $\pi(s)$ could converge before $v^*(s)$
- Do not focus on modeling $R(s, a, s')$ and $T(s, a, s')$
- The agent can influence how it gathers experience so that it is the most useful for learning ($\rightarrow$ exploration/exploitation dilemma)
- Some methods can combine model-based and model-free (e.g., Dyna)

Motivations for Model-free RL (2)

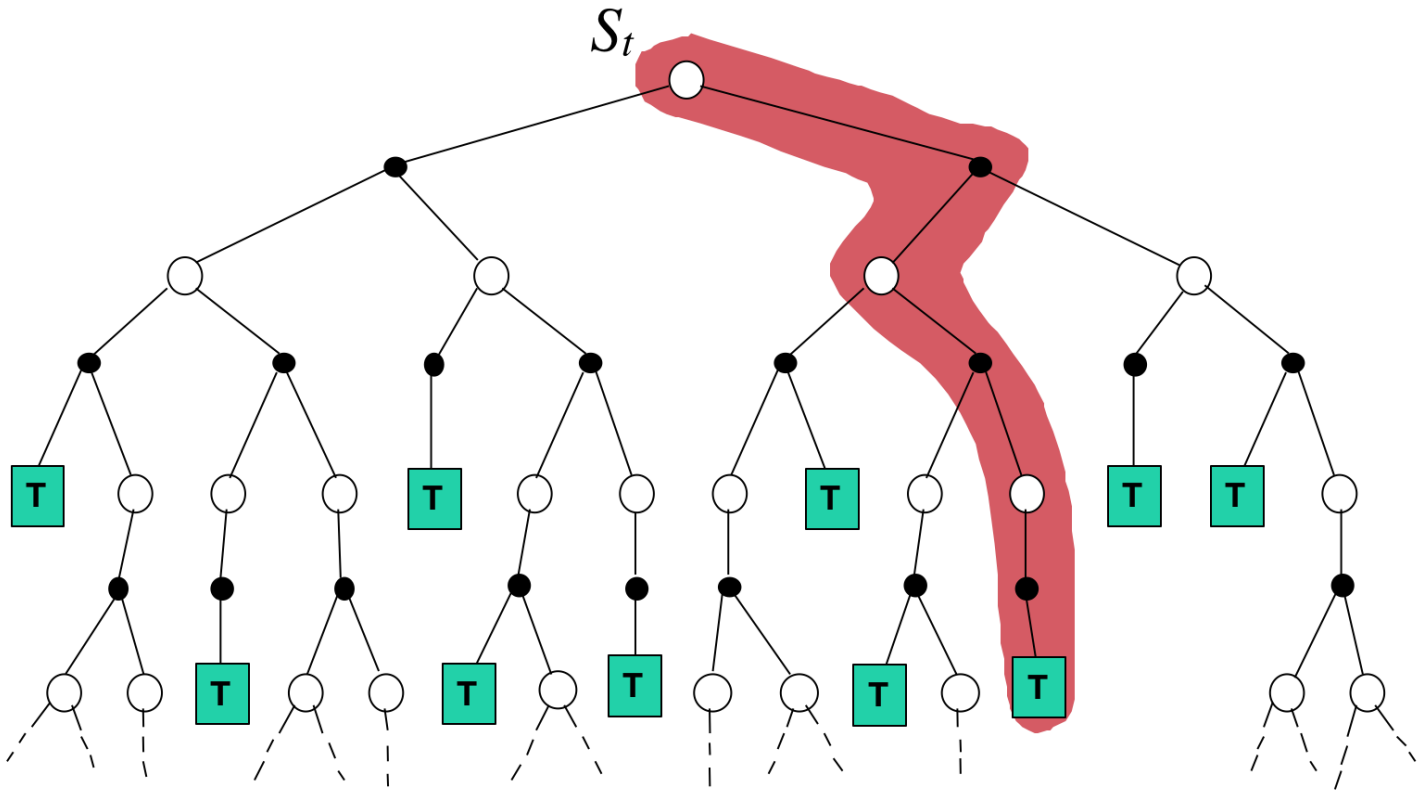
- Some example problems could be modelled as MDPs
  - Control an elevator, a car, a ship, a helicopter, ...
  - Robots : control, walk, play soccer, ...
  - Portfolio management
  - Protein Folding
  - Play (complicated) games
- For most of these problems, either:
  - MDP model is unknown, but experience can be sampled
  - MDP model is known, but is too big to use, except by samples

Model-free RL

- On-policy learning
  - "Learn on the job"
  - Learn about policy π from experience sampled from π
  - Trying to evaluate the policy π at the same time
- Off-policy learning
  - "Look over someone's shoulder"
  - Learn about policy π from experience sampled from other policy

Monte-Carlo methods in model-free RL

- simulate trajectories from each state $s \in \mathcal{S}$ using a given policy π
- estimate $v^{\pi}(s)$ by averaging the observed returns
- starting from $S_t = s$



Monte-Carlo methods

- Value function defined in terms of expected return

$$v^{\pi}(s) = \mathbb{E}^{\pi}[G_t | S_t = s]$$

- Trajectory ("episode" of length T) simulated using a fixed policy π
- During a trajectory, a state S_t can be visited several times
  - e.g., Agent comes twice to the same place in a maze
- Observe the returns at the end of each episode :

$$G_t = \sum_{i=0}^{T} \gamma^i R_{t+i+1}$$

Monte-Carlo methods

- Monte-Carlo principle : use empirical mean instead of $\mathbb{E}^{\pi}[G_t|S_t = s]$ over the number of times k that the state s has been reached

$$\hat{v}_k^{\pi}(s) = \frac{1}{k}\sum_{i=1}^{k} G_i(s)$$

  - requires to store all returns G_i $(i = 1, \ldots, k)$ for each state s
- The average can be computed incrementally

$$\begin{aligned}\hat{v}_k^{\pi} = \frac{1}{k}\sum_{i=1}^{k} G_i &= \frac{1}{k}(G_k + \sum_{i=1}^{k-1} G_i) \\ &= \frac{1}{k}(G_k + (k-1)\hat{v}_{k-1}^{\pi}) \\ &= \hat{v}_{k-1}^{\pi} + \frac{1}{k}(G_k - \hat{v}_{k-1}^{\pi})\end{aligned}$$

  - only need to store the integer k for each state s

Monte-Carlo methods

- Introducing a learning rate $\alpha(S_t)$ instead of $\frac{1}{k}$
- Update the values $v^\pi(S_t)$ for each state S_t visited

$$v^\pi(S_t) \leftarrow v^\pi(S_t) + \alpha(S_t)(G_t - v^\pi(S_t))$$

with the $\alpha(S_t)$ decreasing to zero during learning.

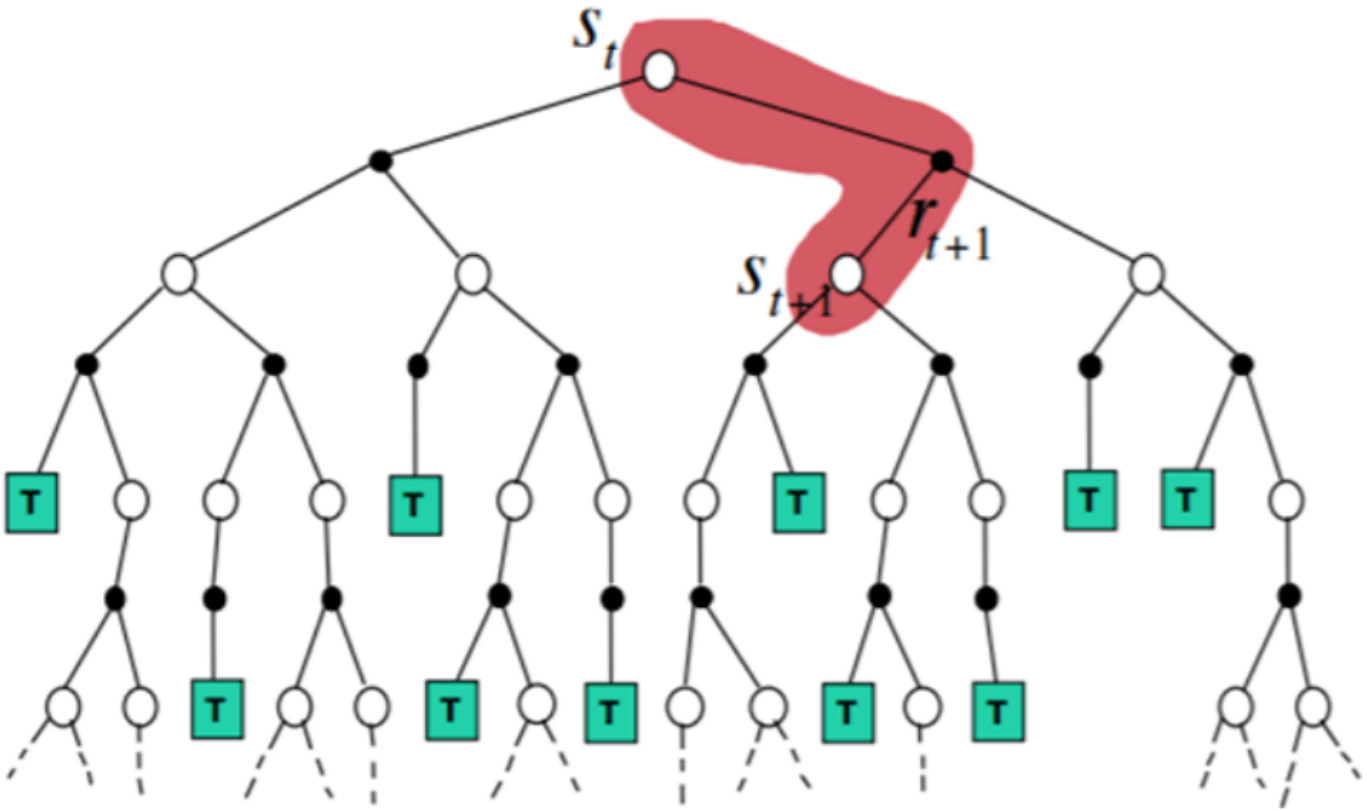
  - no need to store k
  - interesting in non-stationary settings (no need to store past values)
- Guarantee to converge to true average ($\approx$ law of large numbers)
  - α too small : convergence slow
  - α too large : oscillate around average
- These incremental averages occur everywhere in RL

Monte-Carlo methods

- Gives an evaluation of policy π
- Would need exploration to find the best policy π^*
- "every-visit" Monte-Carlo :
  - state s can be updated at each visit during the trajectory
  - the errors are not independent $\rightarrow$ bias
- "first-visit" Monte-Carlo :
  - average of the returns following first visits to s
  - unbiased estimator
- learning has to be decomposed into finite episodes
- have to wait until the end of an episode to learn

Temporal Differences (TD-learning, Rich Sutton, 1988)

- goal : avoid waiting until the end of an episode to learn
- policy π fixed



Temporal Difference TD(0)

How to compute a mean with only one value at a time. . .

- initial choice of $v^\pi(s)$
- recompute $v^\pi(s)$ after each transition (s, a, s', r):

$$[R(s, \pi(s), s') + \gamma v^\pi(s')]$$

- update $v^\pi(s)$ by moving average

$$v^\pi(s) \leftarrow (1 - \alpha)v^\pi(s) + \alpha[R(s, \pi(s), s') + \gamma v^\pi(s')]$$

  where α is the learning rate
- if α small (≈ 0.1) : inject a little bit of new information at each step
- usually α decays over time

Temporal Difference TD(0)

Why is it called TD ?

- the moving average

$$v^\pi(s) \leftarrow (1-\alpha)v^\pi(s) + \alpha[R(s,\pi(s),s') + \gamma v^\pi(s')]$$

can be re-written the following way [temporal difference]

$$v^\pi(s) \leftarrow v^\pi(s) + \alpha \left[\underbrace{[R(s,\pi(s),s') + \gamma v^\pi(s')]}_{\text{new observation = "TD target"}} - \underbrace{v^\pi(s)}_{\text{current estimate}} \right]$$

- TD(0) = one-step TD : update after one transition $(s, \pi(s), s')$
- we update a guess towards a guess = boostrap estimator
- more elaborate methods : TD(λ)...

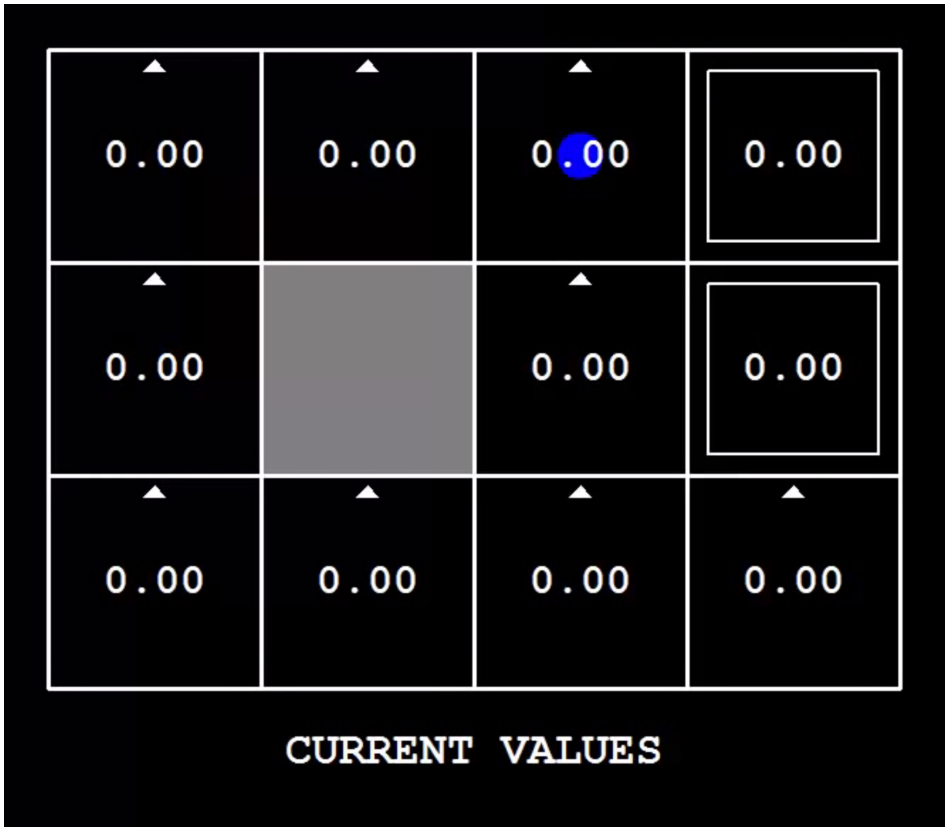
TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$; $\gamma = 1$
- initial state : zeros ...



TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$
- after a few moves : still averaging zeros...



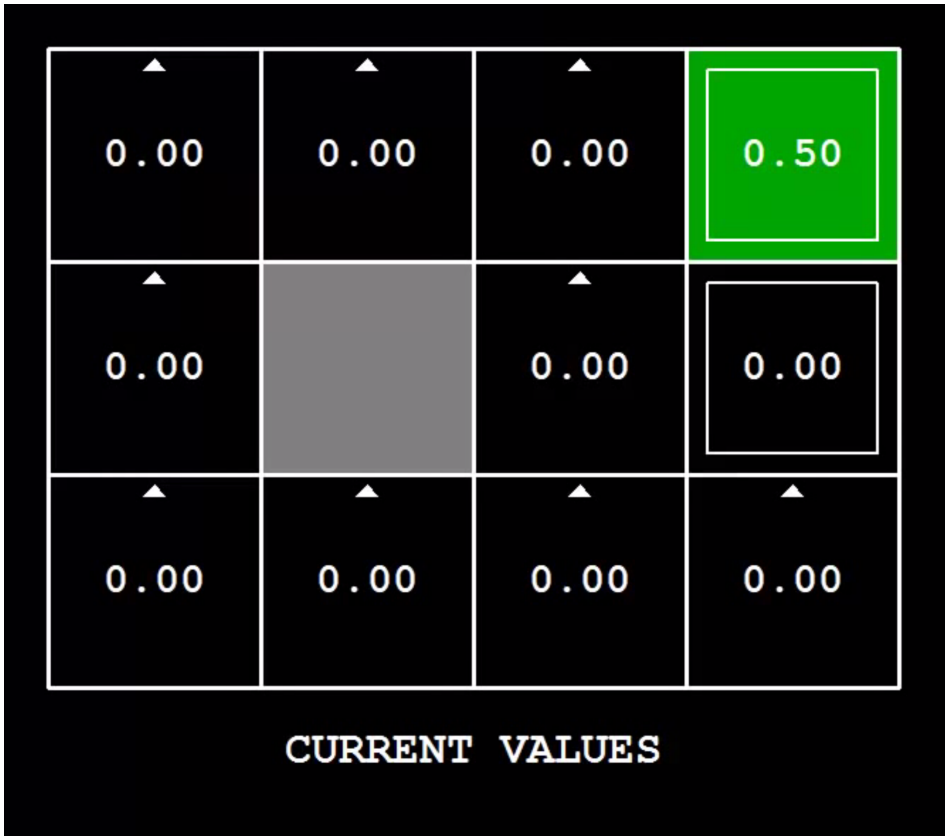
TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$
- one more move : still averaging zeros. . .



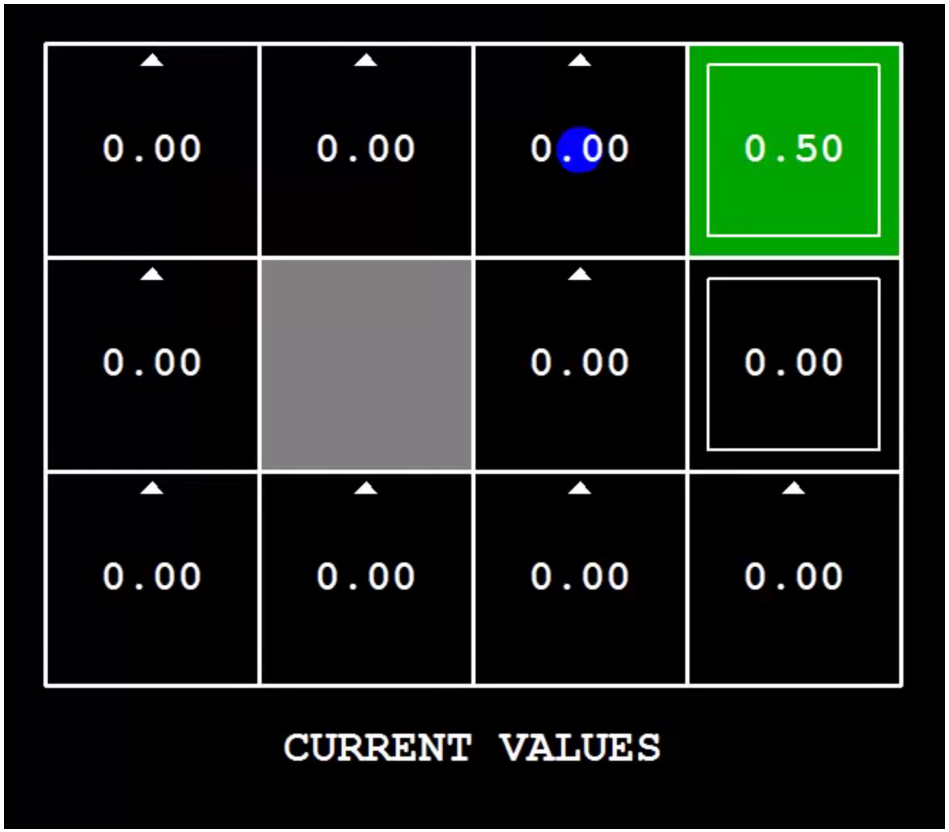
TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$
- expected 0, received $1 \rightarrow 0 + \alpha \times 1 = 0.5$



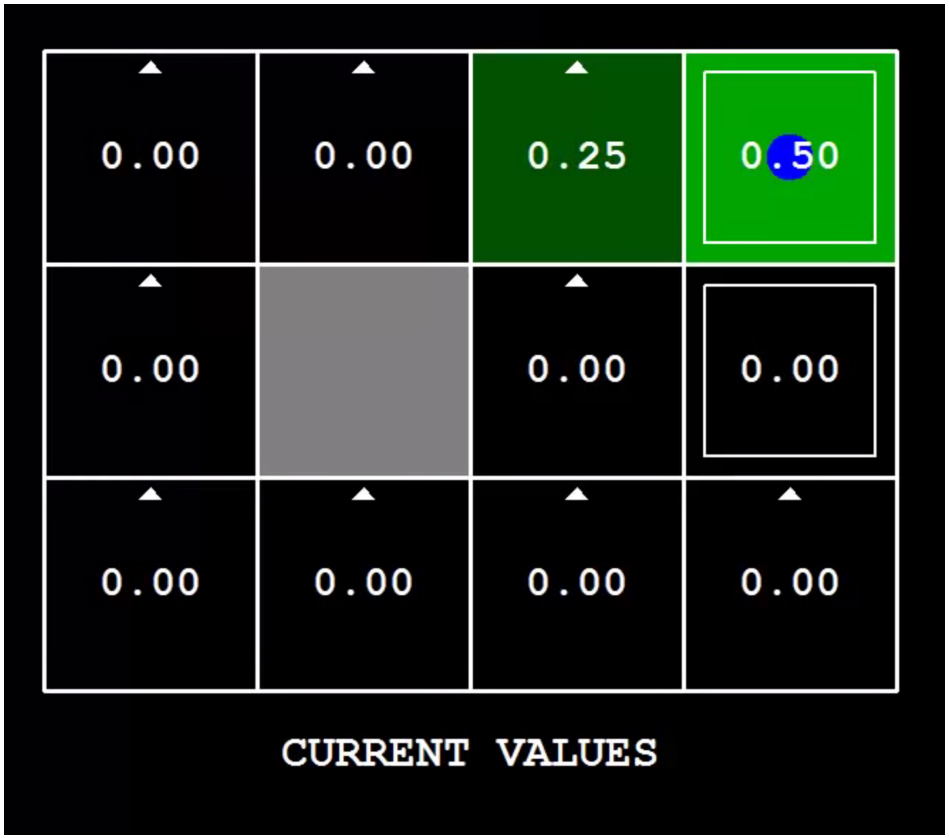
TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$
- start same policy over again



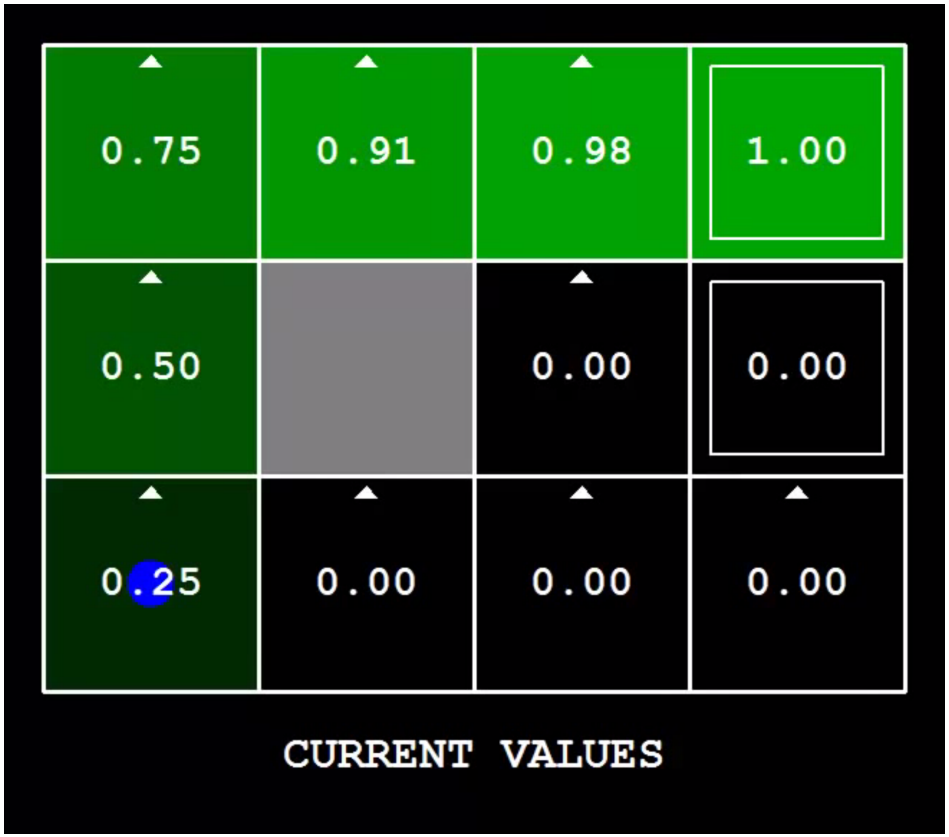
TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$
- expected 0, received 0.5 $\rightarrow 0 + \alpha \times 0.5 = 0.25$



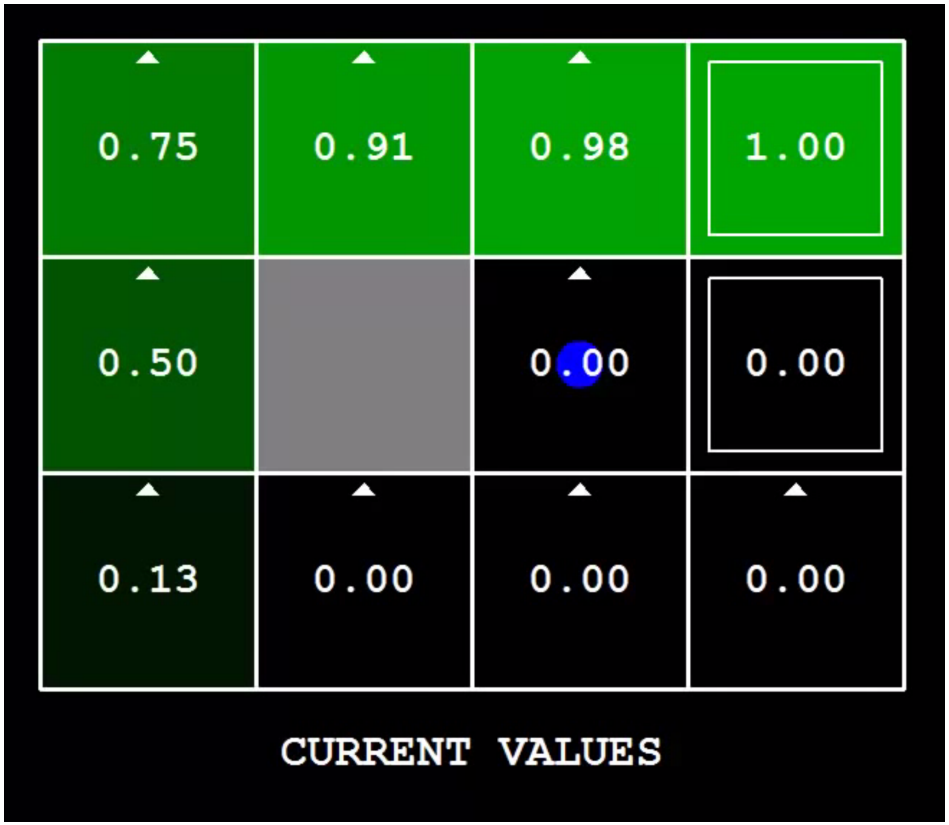
TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$
- later on, following the same policy



TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$
- later on, following a new policy



TD(0) - Illustration

- living reward $= 0$; $\alpha = 0.5$
- received information from $0.98 \rightarrow 0.49$
  - before the end of the episode
  - even if this policy had not been followed before

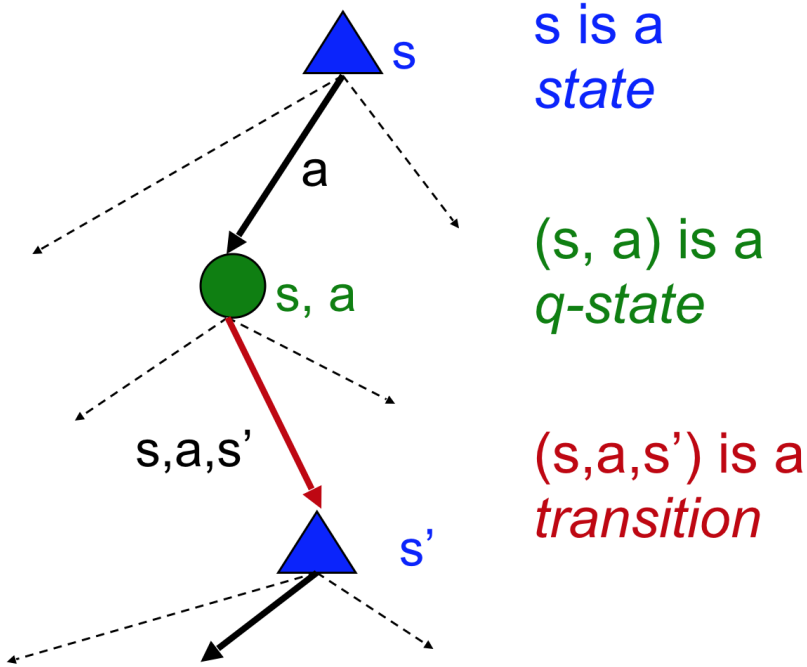


TD(0) - Conclusion

- TD can learn *before* the final outcome
- TD could also learn *without* the final outcome
  - incomplete sequences
  - continuing (non-terminating) environments
- TD exploits the Markov property (which MC does not)
- TD eventually converges to the optimal values

States and q-states

- associate a value $q(s, a)$ to each action in each state
- in q-state : action a has been chosen but not yet executed



Values and q-values

- $v(s) =$ value of a state s

$$v(s) = \mathbb{E}[G_t | S_t = s]$$

- $v^\pi(s) =$ value of a state s for a given policy π

$$v^\pi(s) = \mathbb{E}[G_t | S_t = s, \pi]$$

- $q(s, a) = q$-value of a pair (s, a), called q-state

$$q(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- $q^\pi(s, a) = q$-value of a pair (s, a) for a given policy π

$$q^\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a, \pi]$$

Values and Q-values

Bellman's equations revisited

- by definition,

$$v^*(s) = \max_a q^*(s,a)$$

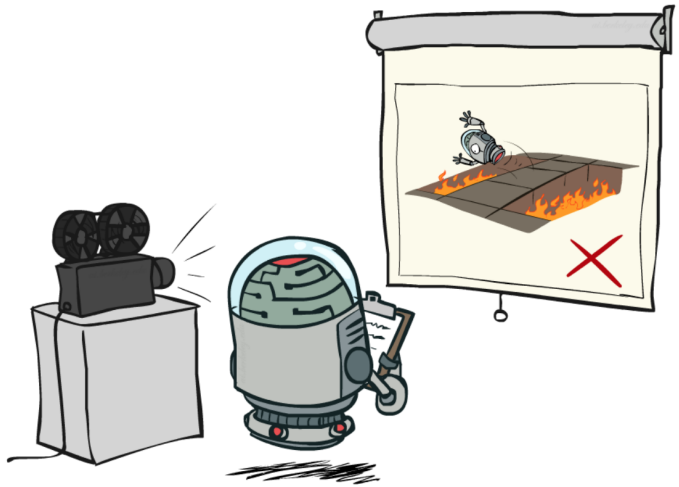
- computation of $q^*(s,a)$:

$$q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma v^*(s')]$$

  - $R(s,a,s')$: reward during the transition
  - $\gamma v^*(s')$: reward after transition

TD(0) revisited

- we only learned about the policies we follow
- we do not really get to choose the actions



Source : ai.berkeley.edu

- we compute $q(s, \pi(s))$ but do not know $q(s, a)\ \forall a$
- therefore we cannot compute $\pi^*(s) = \arg\max_a q(s, a)$
- idea : learn $q(s, a)$ instead of $v(s)$

SARSA

- similar to TD(0) but works on $q(s, a)$ instead of $v(s)$

$$q(s_k, a_k) \leftarrow q(s_k, a_k) + \alpha[r_k + \gamma q(s_{k+1}, a_{k+1}) - q(s_k, a_k)]$$

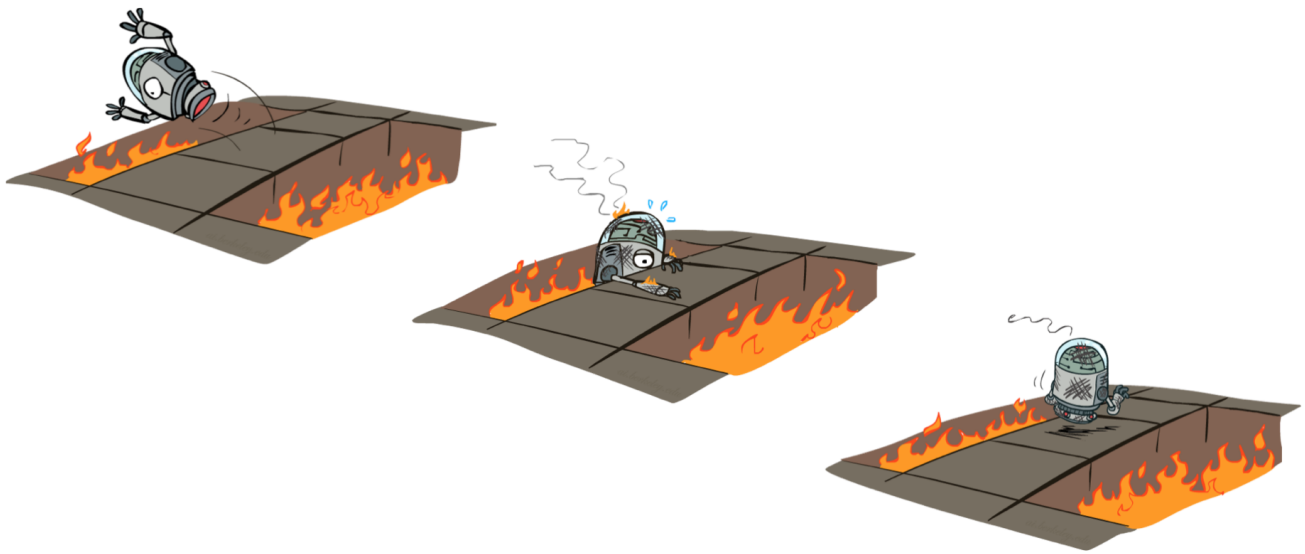
  where α is the learning rate and γ is the discount factor.
- The information necessary for an update is thus the tuple

$$(s_k, a_k, r_k, s_{k+1}, a_{k+1})$$

  hence the name of the technique : SARSA
- implies a "one step look ahead" to determine the action a_{k+1}
- on-policy : actions chosen from the policy followed, which constrains exploration and thus learning
- convergence proofs harder [Singh *et al.*, 2000]

q-learning

- simplification of SARSA
- not necessary to do a "one step look ahead"
- use $\max_a q_k(s_{k+1}, a)$ instead of $q(s_{k+1}, a_{k+1})$
- off-policy, no need to know a_{k+1}



- convergence proved earlier than SARSA [Dayan *et al.*, 1994]

q-value Iteration

- previously: Value Iteration (from $v_0(s) = 0$)

$$v_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma v_k(s')]$$

  - $\max_a$ is not easy to compute from samples

- q-value Iteration (from $q_0(s,a) = 0$)

$$\boxed{q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma \max_{a'} q_k(s',a')]}$$

  - $T(s,a,s') \rightarrow$ compute an average from samples

Analogy with probabilities

- Let us compute the average age of the people ~~in the room~~ online
- If we knew the distribution of the ages $P(a)$

$$\mathbb{E}(A) = \sum_a P(a)a$$

- Unfortunately, $P(a)$ is unknown
- Take a sample of N persons to build a model of $P(a)$

$$\hat{P}(a) = \frac{n(a)}{N} \qquad \Rightarrow \qquad \mathbb{E}(A) \approx \sum_a \hat{P}(a)a$$

- There is really no need to model $P(a)$ explicitely (*model-free*)

$$\mathbb{E}(A) \approx \frac{1}{N} \sum_i a_i$$

- The more frequent ages will naturally appear more often !

q-learning algorithm

- q-learning = stochastic version of q-value Iteration from samples
  - no $T(s, a, s')$, $R(s, a, s')$
  - moving average
- current estimate : $q(s, a)$
- sample (s, a, s', r) (whatever π was used to get a)
- new estimate of $q(s, a)$:

$$[R(s, a, s') + \gamma \max_{a'} q(s', a')]$$

- update $q(s, a)$ by moving average : q-Learning

$$\boxed{q(s, a) \leftarrow (1 - \alpha)q(s, a) + \alpha[R(s, a, s') + \gamma \max_{a'} q(s', a')]}$$

where α is the learning rate

- learn from each sample (s, a, s', r) !

q-learning algorithm

- we can reformulate the moving average update as

$$q(s,a) \leftarrow q(s,a) + \alpha\Delta(s,a)$$

by defining the difference

$$\Delta(s,a) = [R(s,a,s') + \gamma \max_{a'} q(s',a')] - q(s,a)$$

q-learning algorithm

Algorithm 1 q-learning algorithm

Initialize q_0

for $n \in \{0, \ldots, N-1\}$ **do**

$s_n \leftarrow$ chooseState

$a_n \leftarrow$ chooseAction

compute (s'_n, r_n)

update q_n

$$\Delta_n(s_n, a_n) = [R(s_n, a_n, s'_n) + \gamma \max_{a'_n} q(s'_n, a'_n)] - q(s_n, a_n)$$

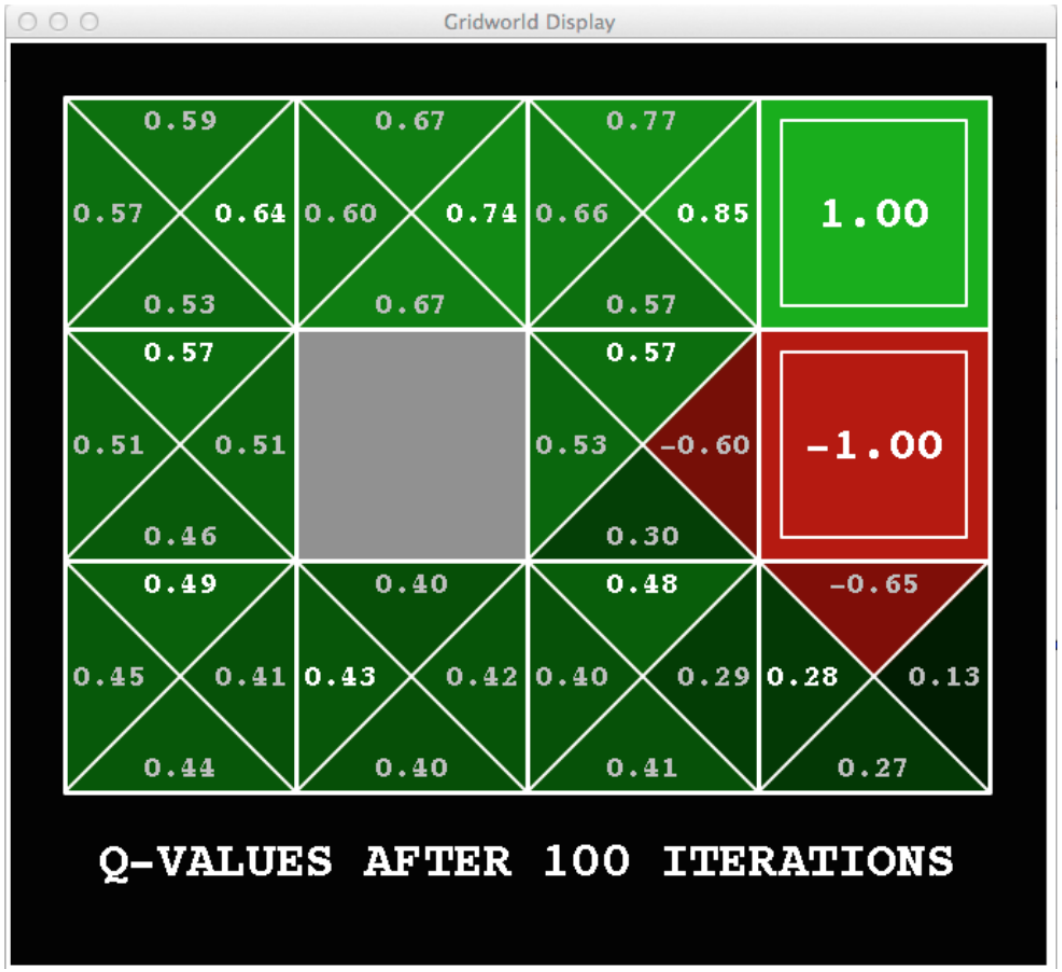
$$q_{n+1}(s_n, a_n) \leftarrow q_n(s_n, a_n) + \alpha \Delta_n(s_n, a_n)$$

end for

return q_N

q-learning algorithm : Illustration

- maintain at each step a lookup table $q(s, a)$
- learn the max



Noise = 0.2
Discount = 0.9
Living reward = 0

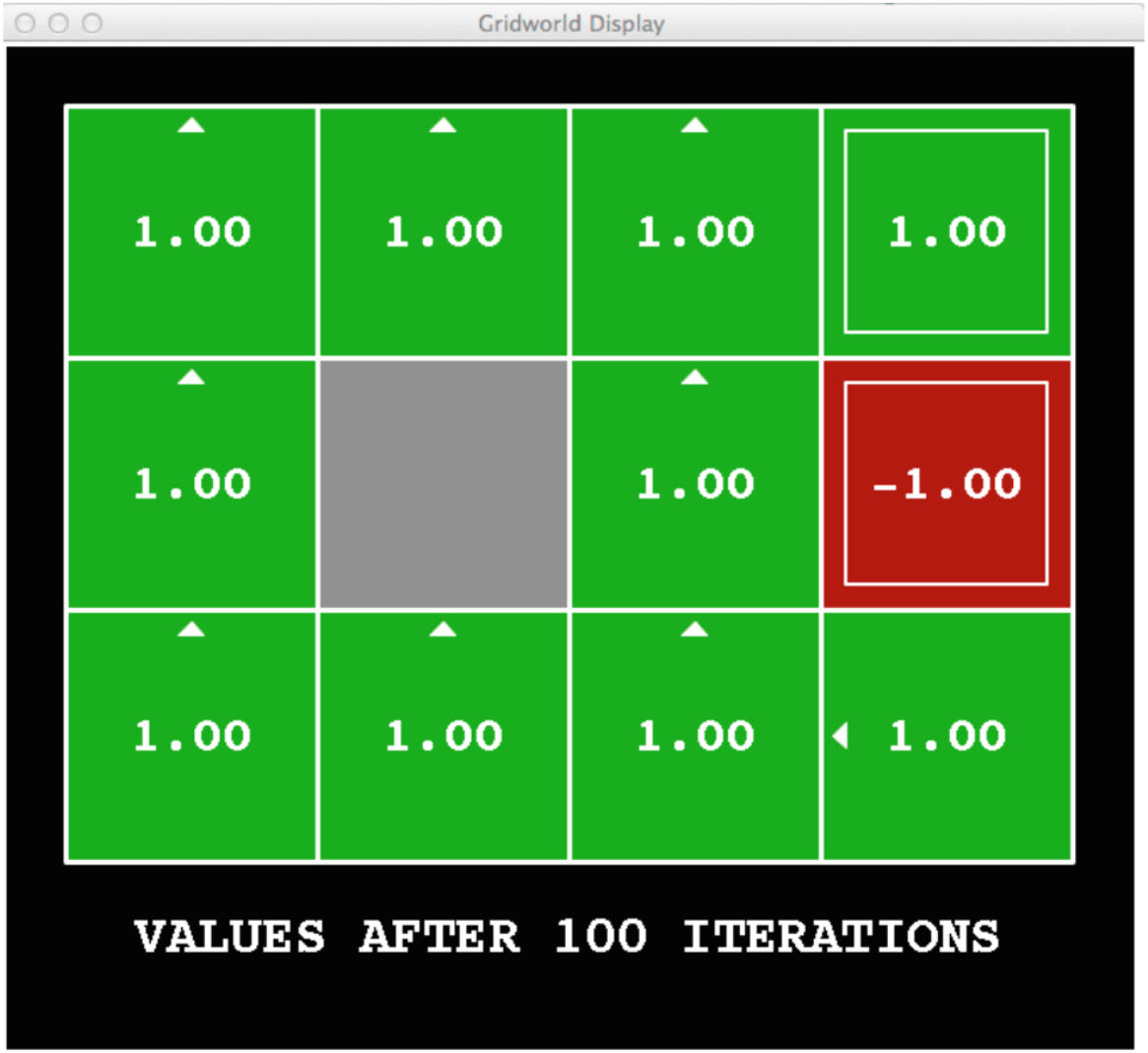
q-learning characteristics

Property : q-learning convergence

$$\lim_{k \to \infty} q_k(s,a) = q^*(s,a)$$

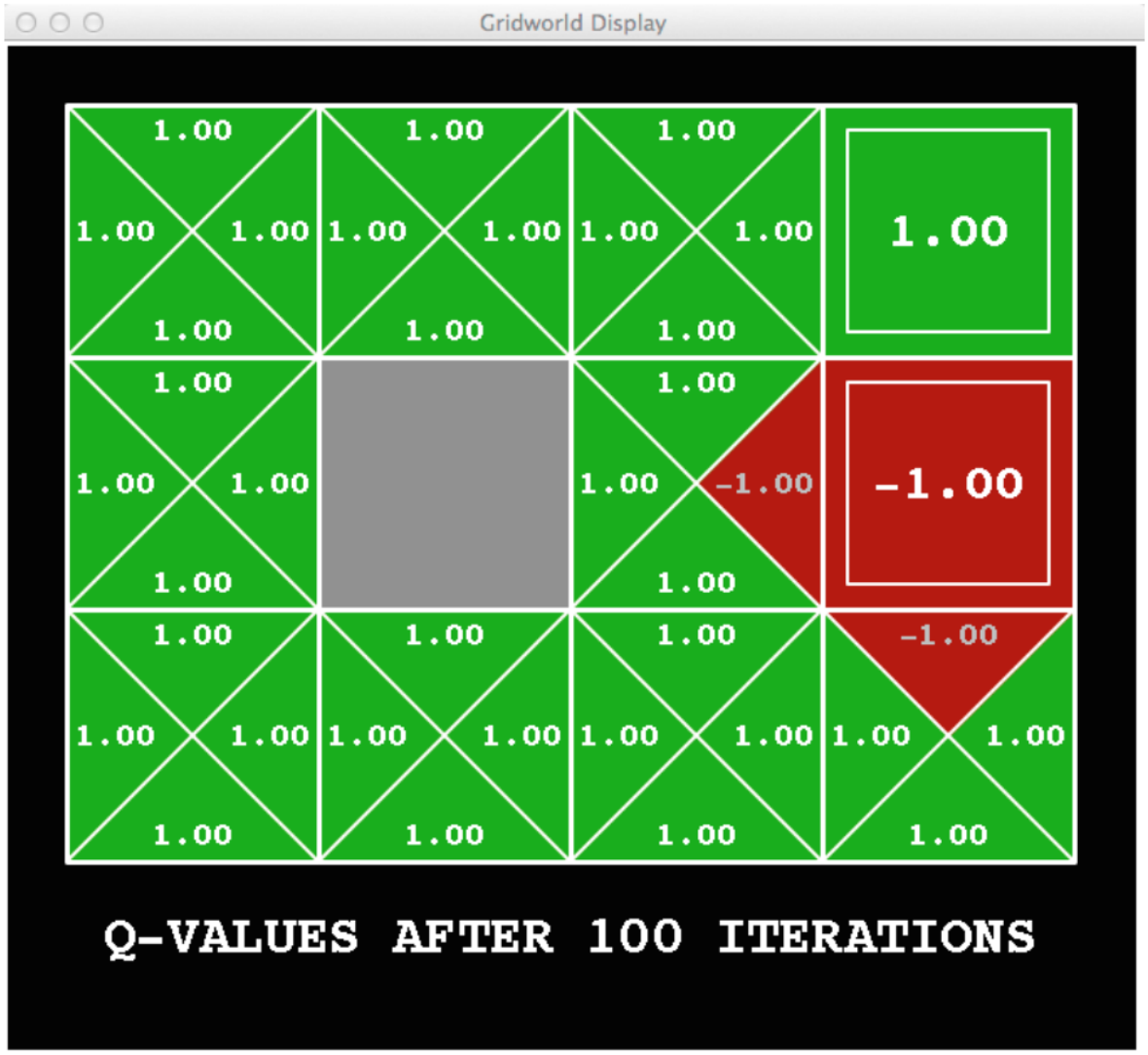
- the convergence can be slow...
  - if $q_0(s) = 0, \forall s$: random walk !
- we learn the optimal policy π^* without even following it
- need α low but not too much ($\alpha \approx 0$ learns nothing)
- policy π used does not matter much
- how to select the actions ?
  - need to explore sufficiently (only learn about the states visited)

Exemple : Grid World



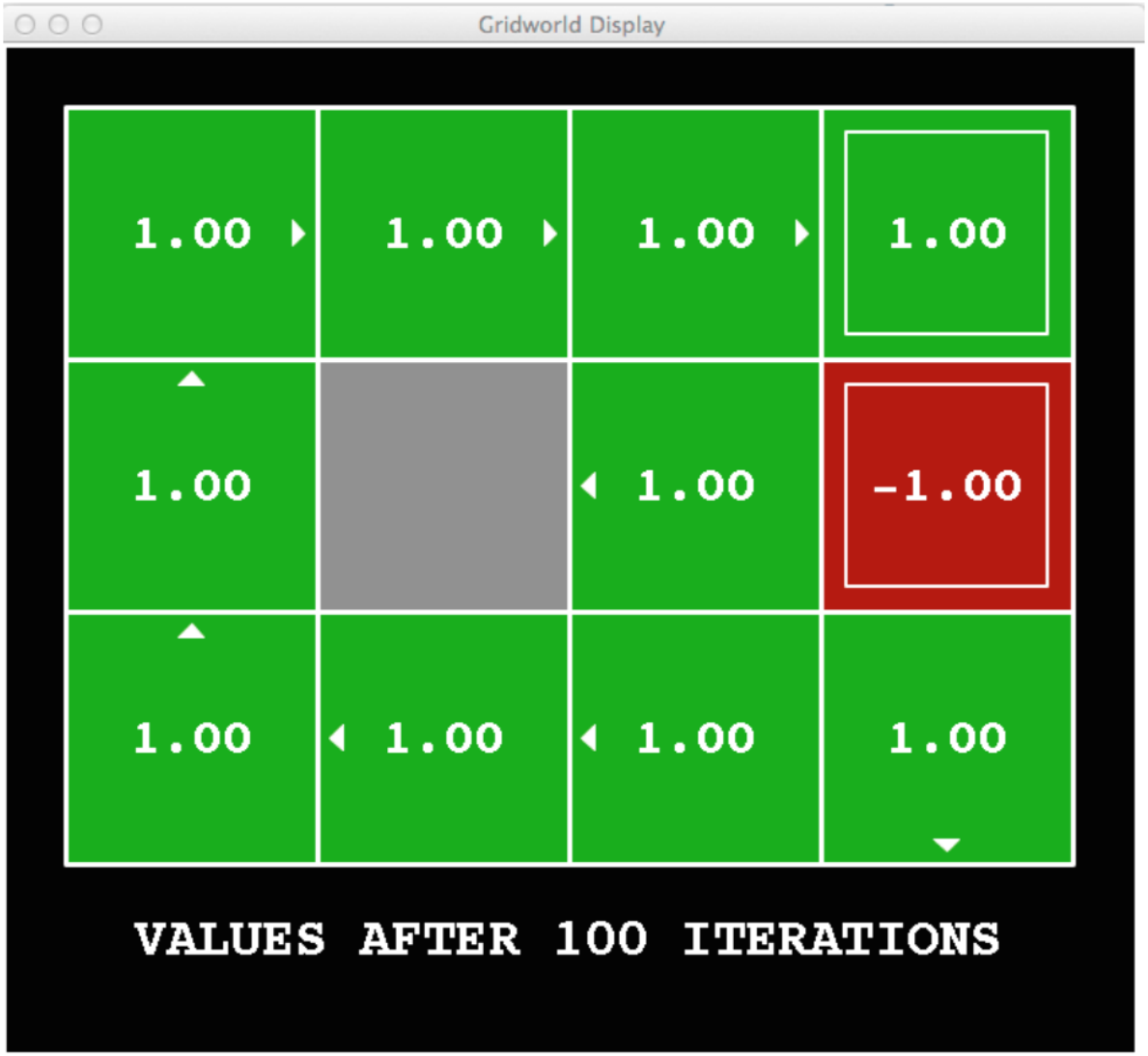
Noise = 0
Discount = 1
Living reward = 0

Exemple : Grid World



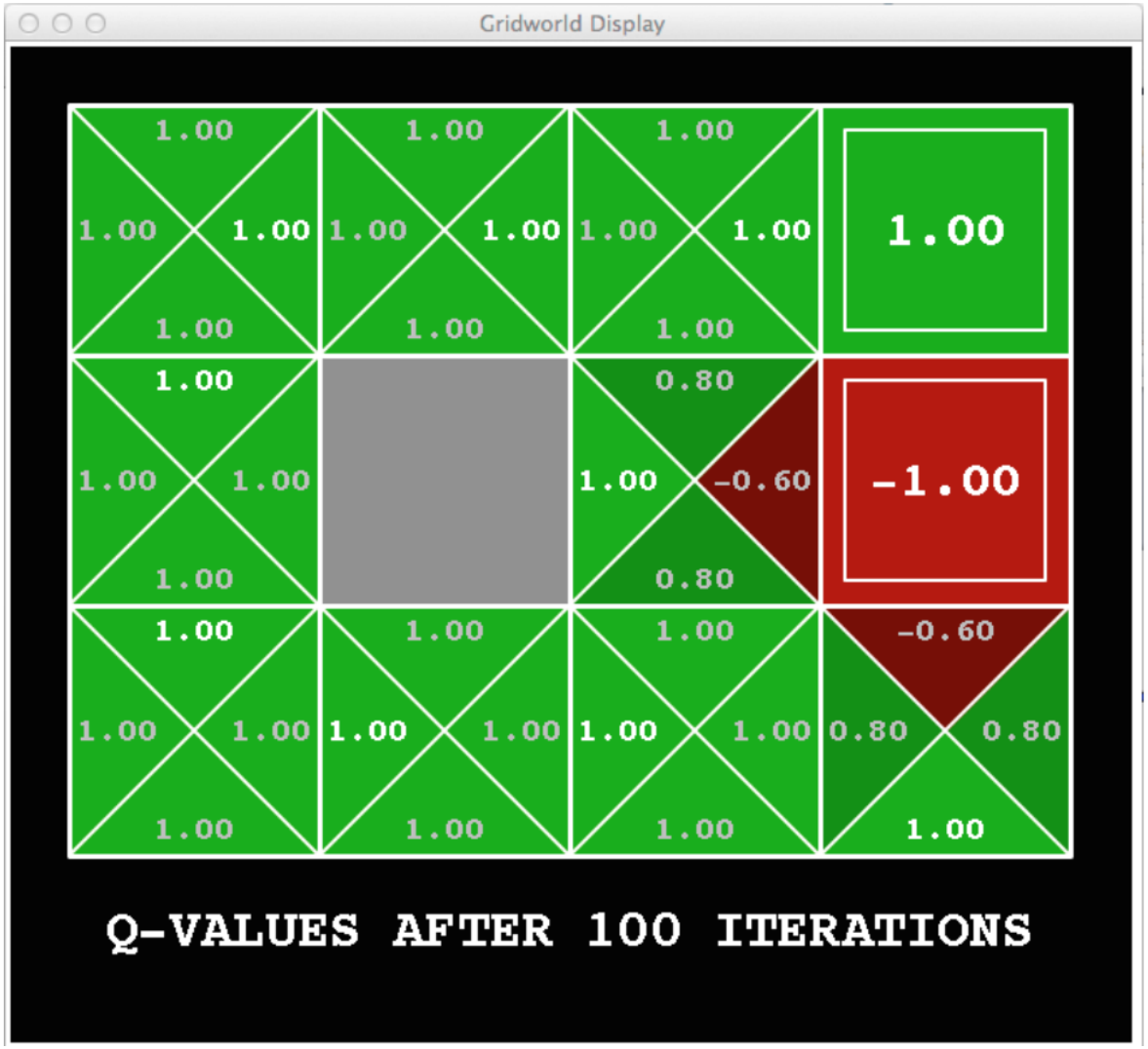
Noise = 0
Discount = 1
Living reward = 0

Exemple : Grid World

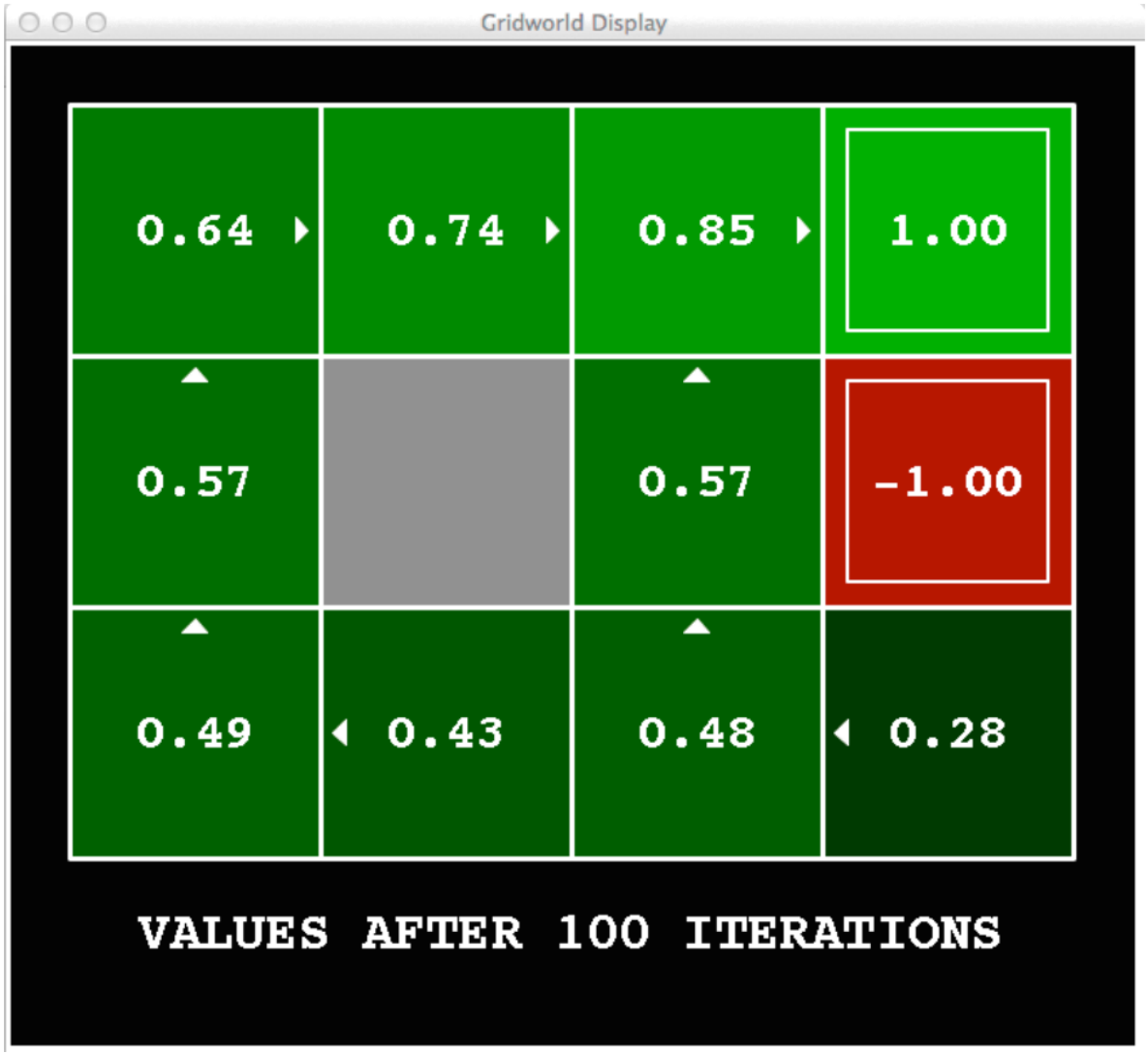


Noise = 0.2
Discount = 1
Living reward = 0

Exemple : Grid World

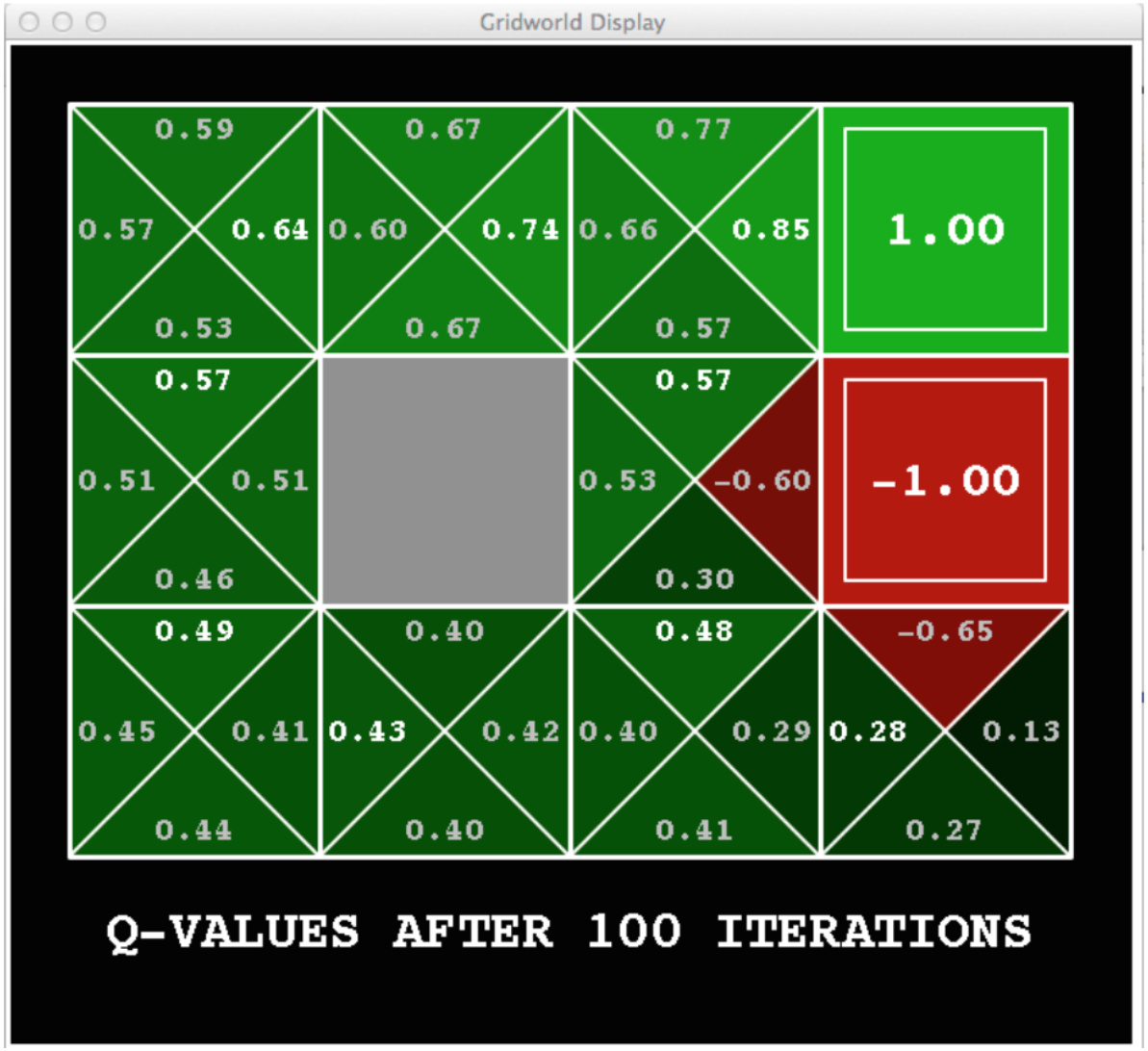

Gridworld Display
1.00
1.00
1.00

Exemple : Grid World



Noise = 0.2
Discount = 0.9
Living reward = 0

Exemple : Grid World



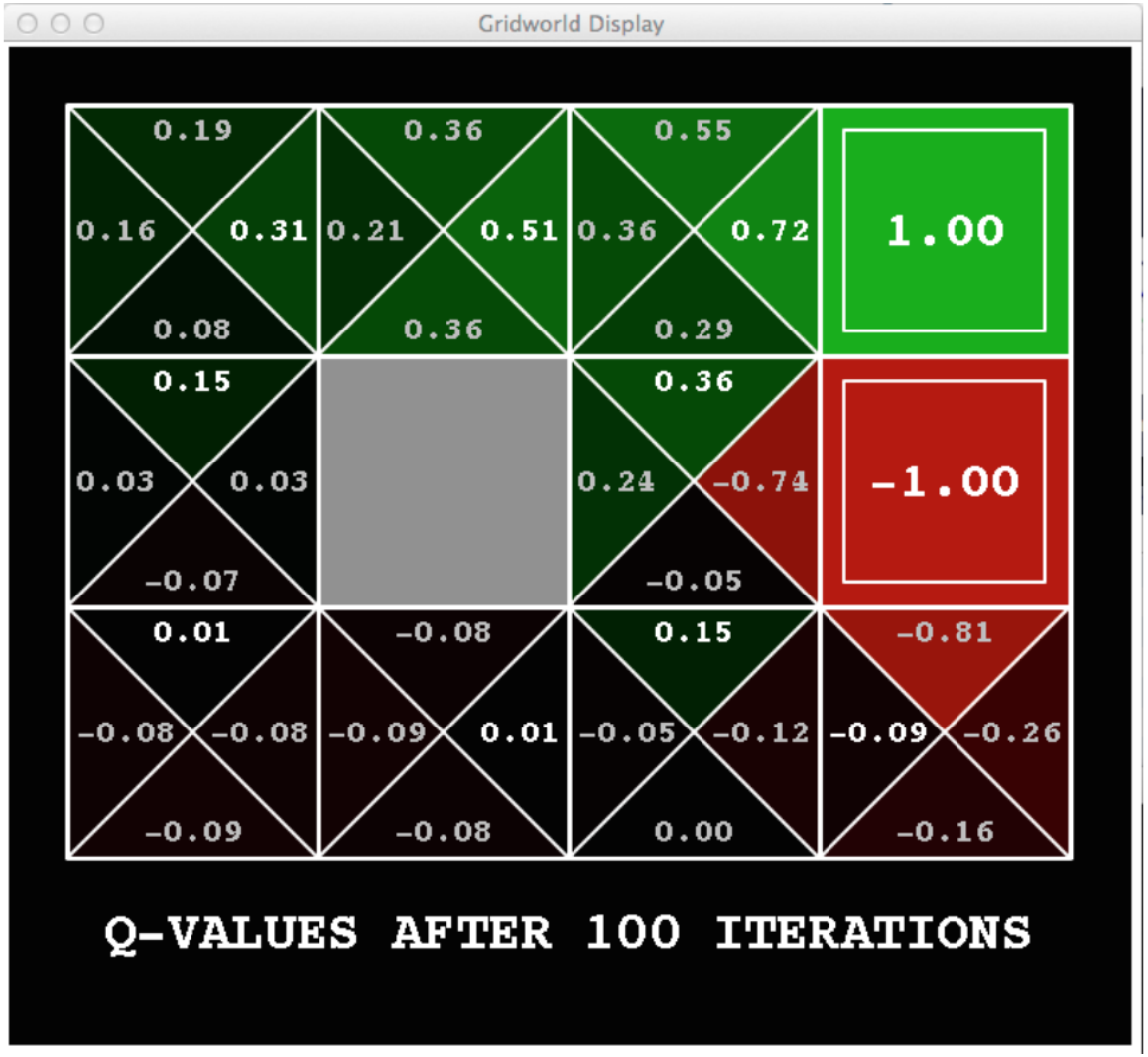
Noise = 0.2
Discount = 0.9
Living reward = 0

Exemple : Grid World



Noise = 0.2
Discount = 0.9
Living reward = -0.1

Exemple : Grid World



Noise = 0.2
Discount = 0.9
Living reward = -0.1

Exploration / Exploitation dilemma

- how to select the actions ?
- greedy policy : at time t, in state s, select a^* such that

$$q_t(s, a^*) = \max_a q_t(s, a)$$

  - exploitation of knowledge on q
  - no exploration
- we might miss important valuable actions

Example of *greedy* action selection



"Behind one door is tenure - behind the other is flipping burgers at McDonald's."

- There are two doors in front of you.
- You open the left door and get reward 0
  $V(left) = 0$
- You open the right door and get reward +1
  $V(right) = +1$
- You open the right door and get reward +3
  $V(right) = +2$
- You open the right door and get reward +2
  $V(right) = +2$

$\vdots$

- Are you sure you've chosen the best door?

Source : David Silver

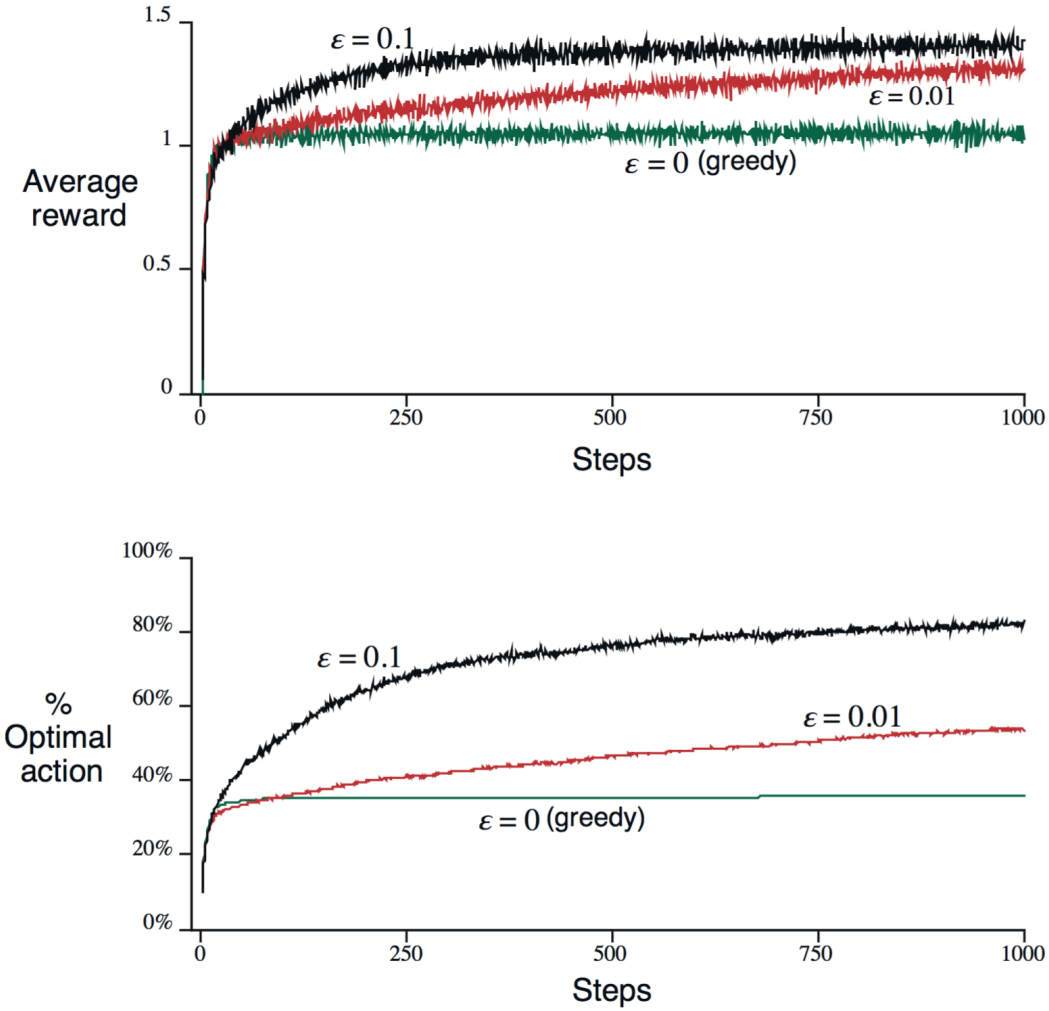
ϵ-*greedy* action selection

- Simplest idea for enforce continual exploration : ϵ-*greedy* policy
  - with probability ϵ, explore a random action
  - with probability $(1 - \epsilon)$, exploit action a^*

  with ϵ not too high ...
- theoretical guarantees that policy gets improved

Greedy vs. *epsilon-greedy* policy

- average performance for multiarmed bandit, on 2000 tasks.



Greedy vs. ϵ-*greedy* policy

- in the previous example
  - *greedy* method only found the optimal action in 1/3 of the cases
    - ★ in the other cases, the first attempts with the optimal action were not satisfying, and the method did not explore that action anymore
  - ϵ-*greedy* method works better because it keeps exploring
- in general, it depends on the variance of the rewards
  - high variance : ϵ-*greedy* better
  - zero variance : *greedy* better, because it only tries each action once and thus finds the solution faster.
- non-stationary rewards: *greedy* will not find the solution

Greedy vs. ϵ-*greedy* policy

- drawbacks of ϵ-*greedy* :
  - explores too many actions in an unstructured way (with the same probability)
  - could spend a lot of time exploring bad actions, and miss quasi-optimal ones
- possible solutions :
  - could diminish ϵ over time
  - give more weight to optimal actions with an exploration function

Exploration function : "softmax" policy

- softmax policy : at time t, pick action a according to Boltzmann (Gibbs) distribution:

$$p(a) = \frac{\exp(\frac{q_t(a)}{\tau})}{\sum_{b=1}^{n} \exp(\frac{q_t(b)}{\tau})}$$

- τ is called "temperature"
  - $\tau \to 0$: greedy
  - $\tau \to \infty$: random
- hard to pick τ without additional knowledge ...

HANDS ON : RL

- do **sections 1.5 → 1.8 of RL1**

